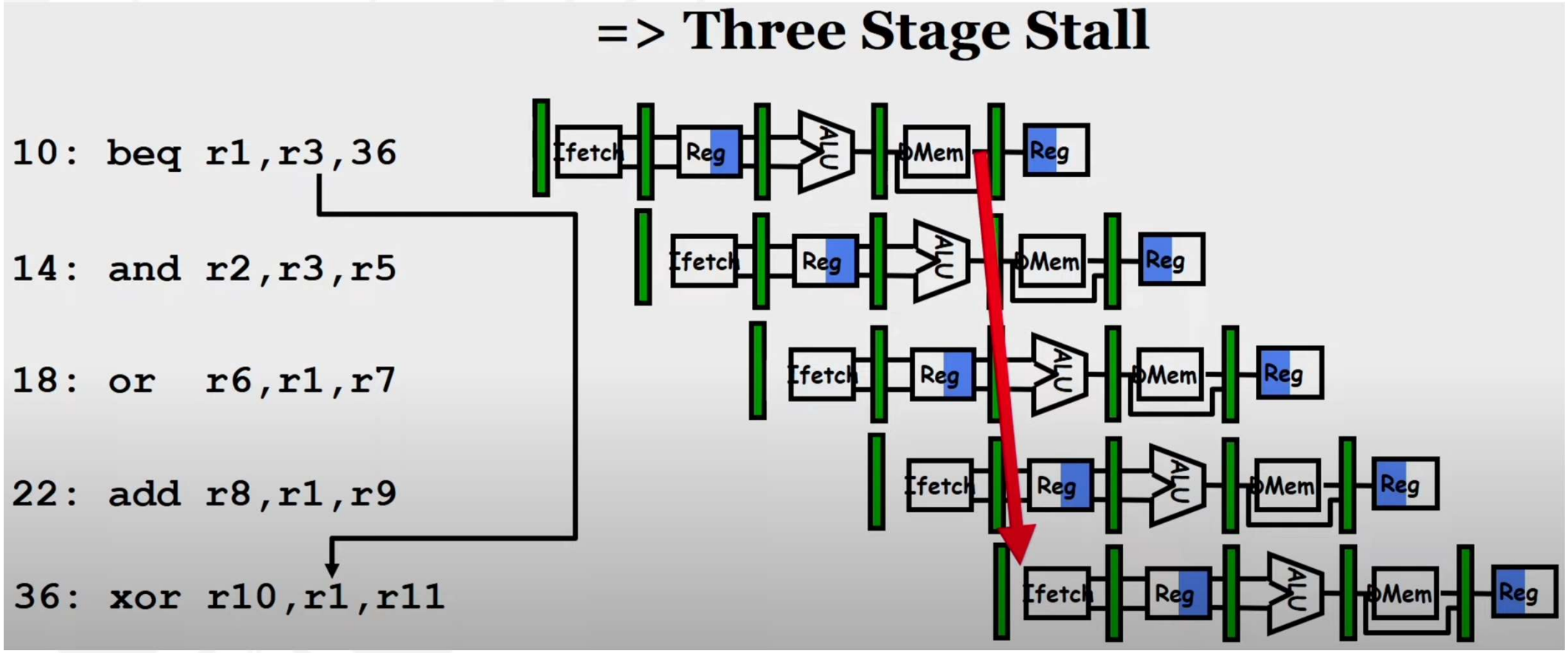
System and Devices (SYS 3)

Lecture 7: Control Hazards and Branch Prediction

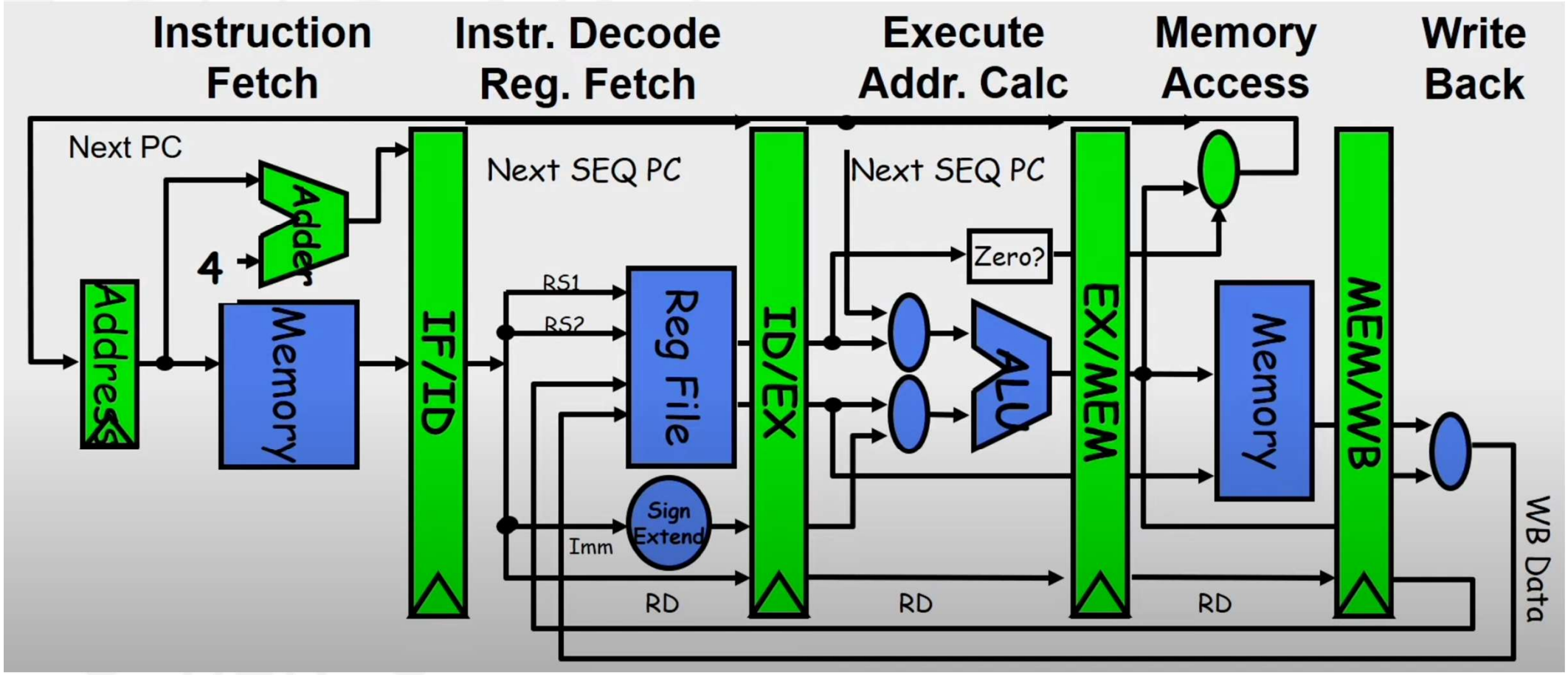
Dr Pengcheng Liu

Control Hazard on Branches



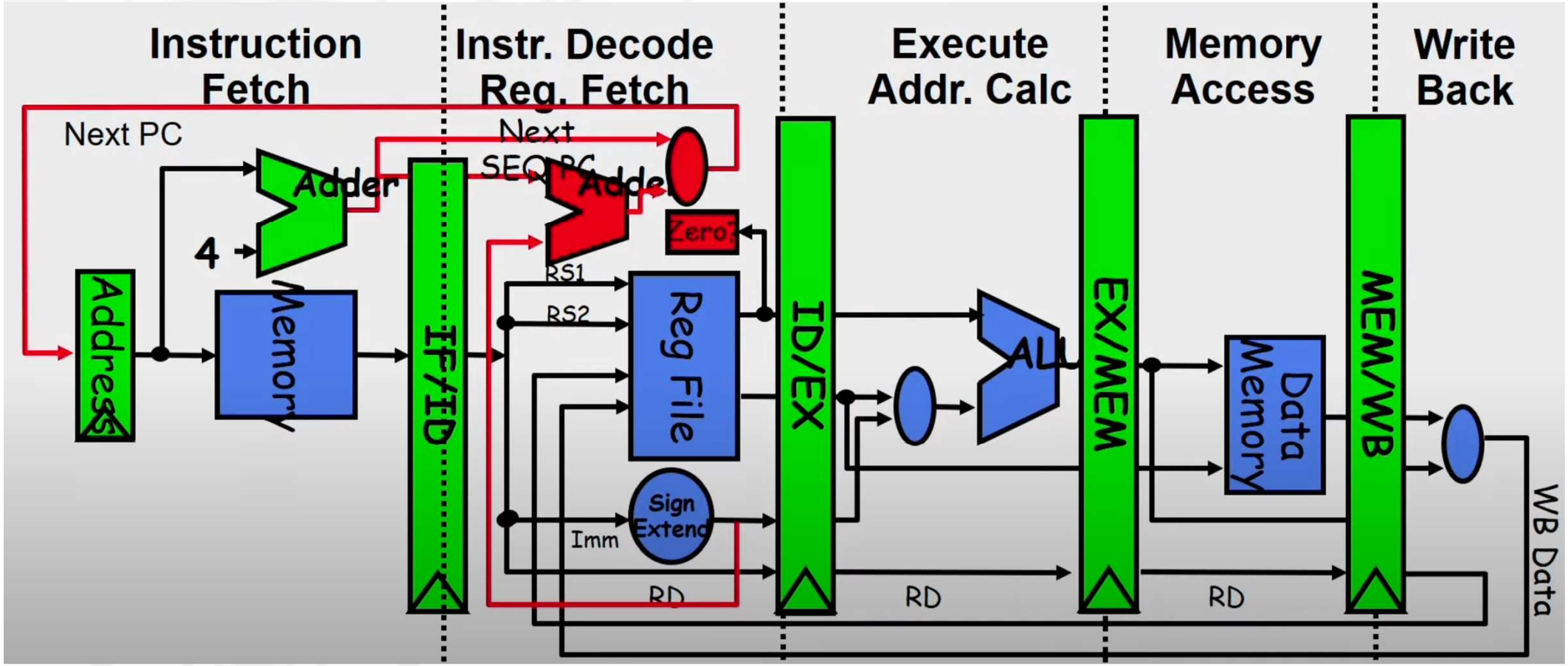
Conventional 5-Stage Pipeline

Branching at 4th Stage



Branch Optimized 5-Stage Pipeline

Branching at 2nd Stage



Four Branch Hazard Alternatives

1: Stall until branch direction is clear

2: Predict Branch Not Taken

3: Predict Branch Taken

4: Delayed Branch

Four Branch Hazard Alternatives

1: Stall until branch direction is clear

2: Predict Branch Not Taken

- Execute successor instructions in sequence
- 'Squash' instruction in pipeline if branch actually taken

Untaken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB

Taken branch instruction	IF	ID	EX	MEM	WB				
Instruction $i + 1$		IF	**idle**	**idle**	**idle**	**idle**			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

The predicted-not-taken scheme and the pipeline sequence when the branch is untaken (top) and taken (bottom). When the branch is untaken, determined during ID, we fetch the fall-through and just continue. If the branch is taken during ID, we restart the fetch at the branch target. This causes all instructions following the branch to stall 1 clock cycle.

Four Branch Hazard Alternatives

3: Predict Branch Taken

- But branch target address is not known by IF stage
- Target is known at same time as branch outcome (ID stage)
- 1 cycle branch penalty will still be incurred

Four Branch Hazard Alternatives

4: Delayed Branch

- Define branch to take place AFTER one instruction following the branch instruction
- 1 slot delay allows proper decision and branch target address in 5 stage pipeline
- Where to get instructions to fill branch delay slot?

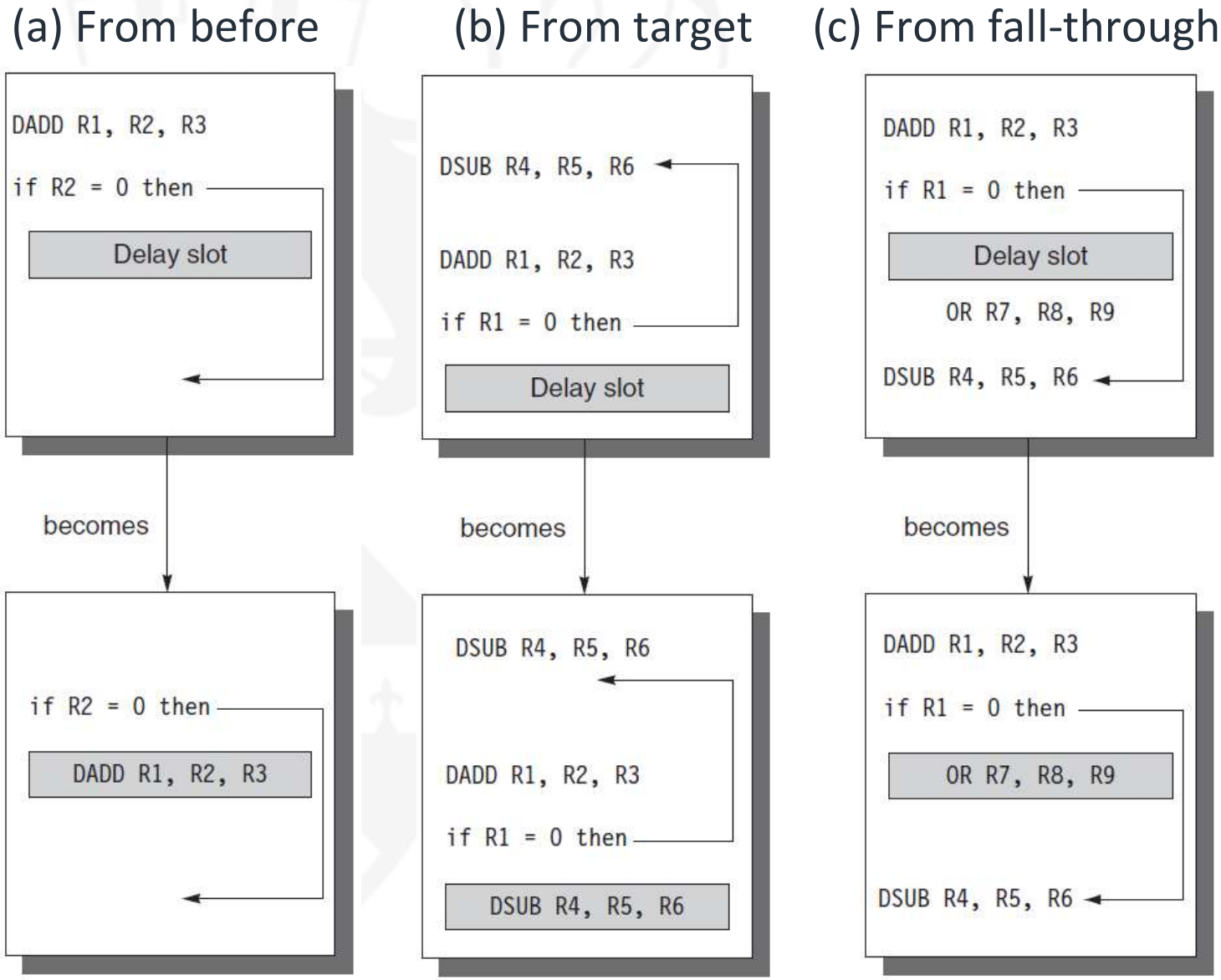
Untaken branch instruction	IF	ID	EX	MEM	WB				
Branch delay instruction ($i + 1$)		IF	ID	EX	MEM	WB			
Instruction $i + 2$			IF	ID	EX	MEM	WB		
Instruction $i + 3$				IF	ID	EX	MEM	WB	
Instruction $i + 4$					IF	ID	EX	MEM	WB

Taken branch instruction	IF	ID	EX	MEM	WB				
Branch delay instruction ($i + 1$)		IF	ID	EX	MEM	WB			
Branch target			IF	ID	EX	MEM	WB		
Branch target + 1				IF	ID	EX	MEM	WB	
Branch target + 2					IF	ID	EX	MEM	WB

The behavior of a delayed branch is the same whether or not the branch is taken. The instructions in the delay slot (there is only one delay slot for MIPS) are executed. If the branch is untaken, execution continues with the instruction after the branch delay instruction; if the branch is taken, execution continues at the branch target.

When the instruction in the branch delay slot is also a branch, the meaning is unclear: If the branch is not taken, what should happen to the branch in the branch delay slot? Because of this confusion, architectures with delay branches often disallow putting a branch in the delay slot.

Filling Branch Delay Slot



Scheduling the branch delay slot. The top box in each pair shows the code before scheduling; the bottom box shows the scheduled code. In (a), the delay slot is scheduled with an independent instruction from before the branch. This is the best choice. In (b), the branch delay slot is scheduled from the target of the branch; usually the target instruction will need to be copied because it can be reached by another path. Strategy (b) is preferred when the branch is taken with high probability, such as a loop branch. Finally, the branch may be scheduled from the not-taken fall-through as in (c).

Conditional Branches

- **When do you know you have a branch?**

  - During ID cycle (Could you know before that)

- **When do you know if the branch is Taken or Not-Taken?**

  - During EXE cycle/ ID stage depending on the design

- We need for sophisticated solutions for following cases

  - Modern pipeline are deep (10+ stages)
  - Several instructions issued/cycle
  - Several predicted branches in-flight at the same time

Dynamic Branch Prediction

- **Execution of a branch requires knowledge of:**

  - Branch instruction: encode whether instruction is a branch or not. Decide on taken or not taken (i.e., prediction can be done at IF stage)

  - Whether the branch is Taken/Not-Taken (hence a branch prediction mechanism)

  - If the branch is taken what is the target address (can be computed but can also be 'precomputed', i.e., stored in some table)

  - If the branch is taken what is the instruction at the branch target address (saves the fetch cycle for that instruction)

Dynamic Branch Prediction

- **Use a Branch Prediction Buffer (BPB)**

  - Also called Branch Prediction Table (BPT), Branch History Table (BHT)

  - Records previous outcomes of the branch instruction

  - How to index into the table is an issue

- **A prediction using BPB is attempted when the branch instruction is fetched (IF stage or equivalent)**

- **It is acted upon during ID stage (when we know we have a branch)**

Dynamic Branch Prediction

- **Has a prediction been made (Y/N)**

  - If not use default 'Not Taken'

- Is it correct or incorrect?

- Two cases:

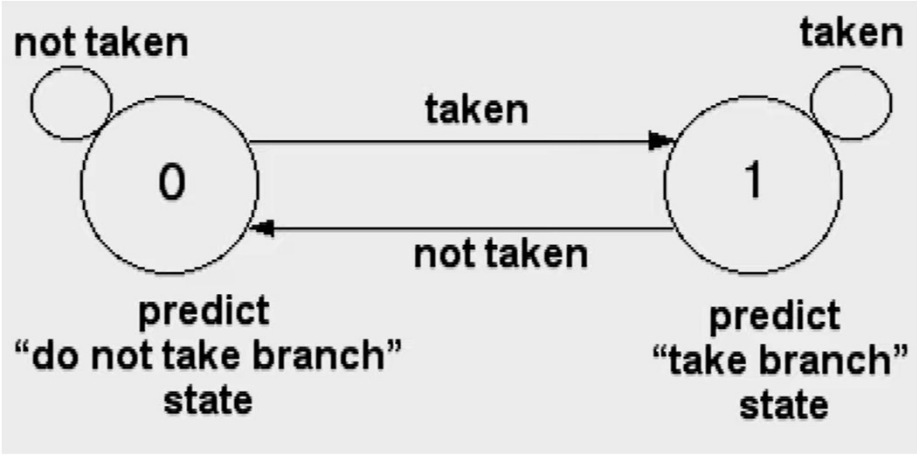
  - Case 1: Yes and the prediction was correct (known at ID stage) or Not but the default was correct: No delay

  - Case 2: Yes and he prediction was incorrect or No and the default was incorrect: Delay

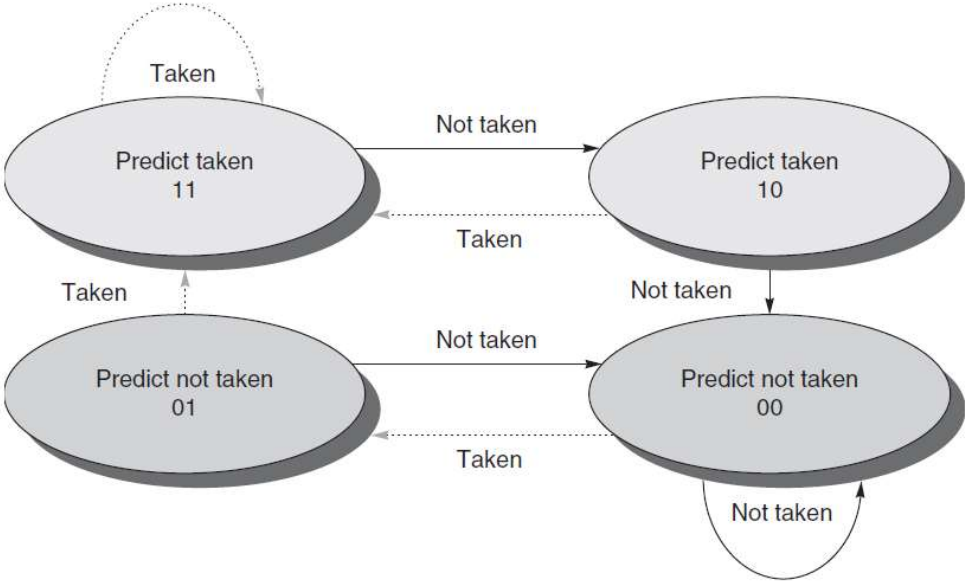
Prediction Scheme with 1 or 2 bit FSM

- The use of a 2-bit predictor will allow branches that favor taken (or not taken) to be mispredicted less often than the one-bit case.

(reinforcement learning)



The states in a 2-bit prediction scheme. By using 2 bits rather than 1, a branch that strongly favors taken or not taken—as many branches do—will be mispredicted less often than with a 1-bit predictor. The 2 bits are used to encode the four states in the system. The 2-bit scheme is actually a specialization of a more general scheme that has an n-bit saturating counter for each entry in the prediction buffer. With an n-bit counter, the counter can take on values between 0 and 2n – 1: When the counter is greater than or equal to one-half of its maximum value (2n – 1), the branch is predicted as taken; otherwise, it is predicted as untaken. Most systems rely on 2-bit branch predictors rather than the more general n-bit predictors.



Branch Prediction In Hardware

- **Branch prediction is extremely useful in loops**
- A simple branch prediction can be implemented using a small amount of memory indexed by lower order bits of the address of the branch instruction. (branch prediction buffer)
- One bit stores whether the branch was taken or not.
- The next time the branch instruction is fetched refer this bit.

BPB

2000	1011

Advanced Branch Prediction Techniques

- **Basic 2-bit predictor**

  - For each branch: Predict T or NT

  - If the prediction is wrong for two consecutive times, change prediction

- Correlating predictor

  - Multiple 2-bit predictors for each branch

  - One for each possible combination of outcomes of preceding n branches

```
if ( x = = 2)        /* br-1*/
        x = 0;
if ( y = = 2)        /* br-2*/
        y = 0;
if ( x != y)         /* br-3/
            do this
else do that
```

Advanced Branch Prediction Techniques

- **Local predictor**

  - Multiple 2-bit predictors for each branch

  - One for each possible combination of outcomes for the last n occurrences of this branch

- **Tournament predictor**

  - Combine correlating predictor with local predictor

Branch-Target Buffer

- To reduce the branch penalty, know whether the as-yet-undecoded instruction is a branch. If so, what the next program counter (PC) should be.
- If the instruction is a branch and we know what the next PC should be, we can have a branch penalty of zero.
- A branch-prediction cache that stores the predicted address for next instruction after a branch is called a branch-target buffer or branch-target cache.

2000: If (a = b) then 6000

.

.

6000: R3 = R4 + R6

PC

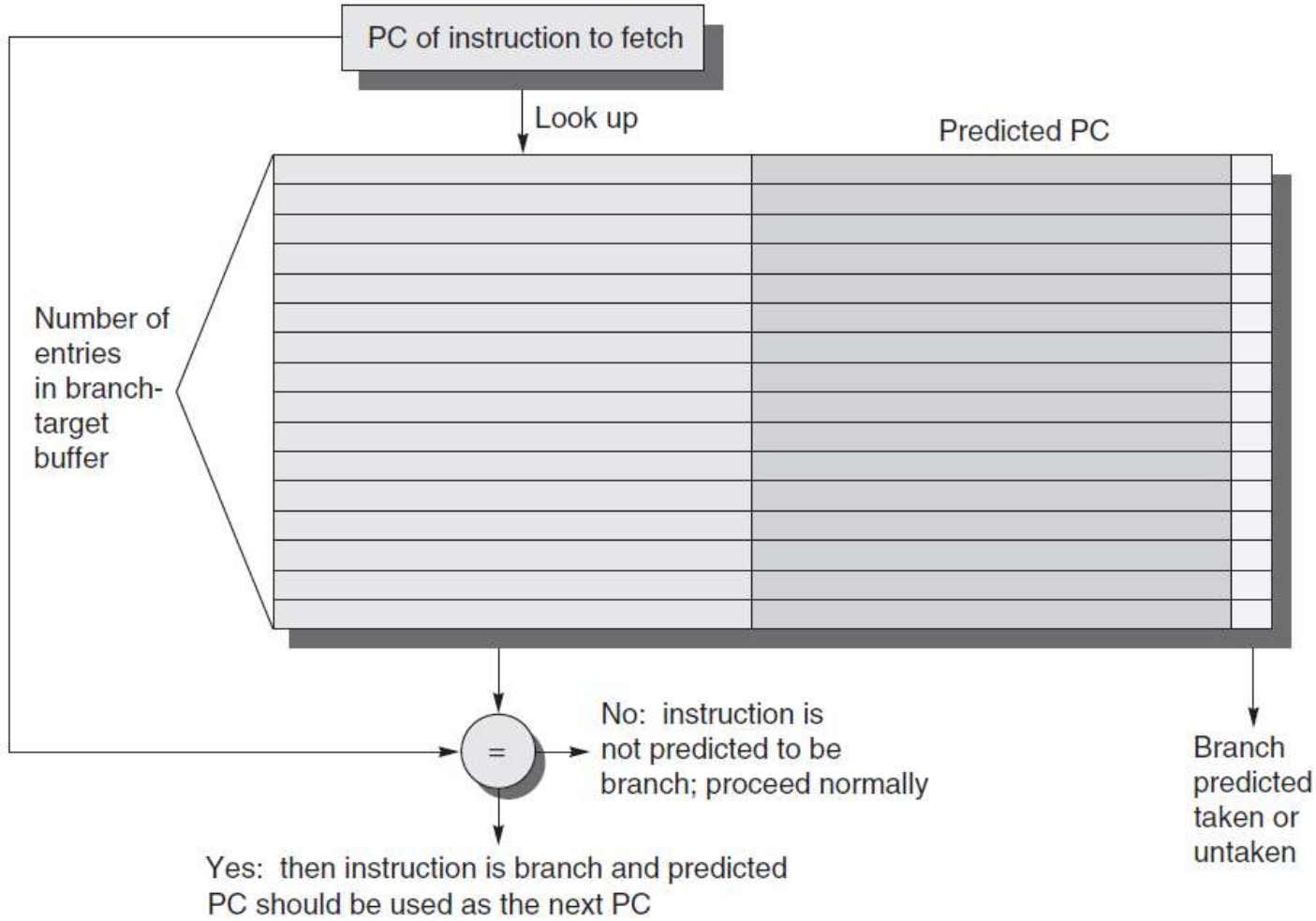
2000

	1100	

BTB

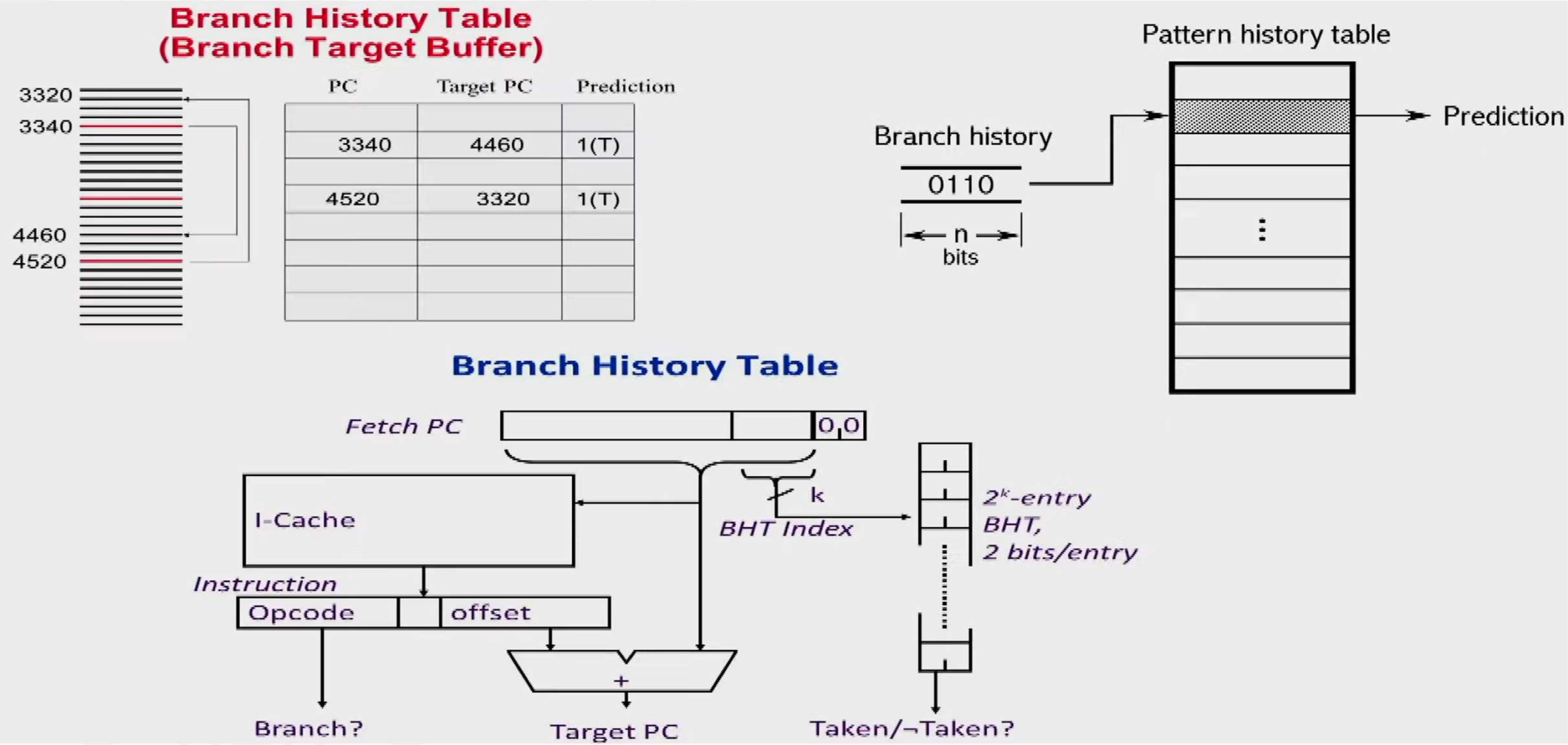
2000	1100	R3=R4+R6

Branch-Target Buffer



A branch-target buffer. The PC of the instruction being fetched is matched against a set of instruction addresses stored in the first column; these represent the addresses of known branches. If the PC matches one of these entries, then the instruction being fetched is a taken branch, and the second field, predicted PC, contains the prediction for the next PC after the branch. Fetching begins immediately at that address. The third field, which is optional, may be used for extra prediction state bits.

Branch-Target Buffer



4K-entry BHT, 2 bits/entry, about 80-90% correct predictions

Branch Folding

- Optimization on BTB to make zero cycle branch

  - Larger branch-target buffer: Store one or more target instructions

  - Add target instruction into BTB to deal with longer decoding time required by larger buffer

  - Branch folding can be used to obtain 0-cycle unconditional branches and sometimes 0-cyle conditional branches.

Summary

Suggested Readings

Computer Architecture: A Quantitative Approach (5th Edition)
by John Hennessy and David A Patterson:

- pp. C-21 - C-30: Branch Hazards
- pp. 162-167: 3.3 Reducing Branch Costs with Advanced Branch Prediction
- pp. 203-206: Branch-Target Buffers